МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)

Факультет «Информатика и вычислительная техника»

Кафедра «Математика и информатика»

ЛАБОРАТОРНАЯ РАБОТА №3

Тема: «МНОЖЕСТВЕННОЕ НАСЛЕДОВАНИЕ»

Дисциплина (модуль) «Программирование»

Направление подготовки/специальность 09.03.02 Информационные системы и технологии

Направленность (профиль) Искусственный интеллект и математическое моделирование

Номер варианта 2 Группа ВИИМ21

Обучающаяся _____ Шевченко О.Н.
подпись, дата

Лабораторную работу проверил(а) _____ доц. Н. Д. Панасенко
подпись, дата

Ростов-на-Дону
2026

Цель работы: освоить принципы множественного наследования в Python, понять механизм MRO (Method Resolution Order), научиться проектировать классы с использованием нескольких родительских классов и разрешать потенциальные конфликты методов.

Задание

1. Написать классы по описанию варианта 2 «Сотрудники компании». Базовый Employee содержит уникальный id. Классы ManagerSkills и DeveloperSkills реализуют work; Manager наследует Employee и ManagerSkills, Developer — Employee и DeveloperSkills. Создать общий список сотрудников и вызвать work в цикле. Объяснить возможную трудность множественного наследования и способы её разрешения.

2. Для направления 09.03.02 «Информационные системы и технологии», профиля «Искусственный интеллект и математическое моделирование», выполнить вариант 2 «Слои нейросети»: Layer с полем id; DenseLayer и ConvolutionalLayer с методами forward и backward; классы Dense и Conv с множественным наследованием от Layer и соответствующего класса поведения.

Выполнить также общие требования задания 2: разработать прототип информационно-аналитического ядра. Предусмотреть базовый класс с id и metadata; миксины валидации, кэширования, журналирования и экспорта; MLModel, EconomicIndicator и HybridAnalyzer. Продемонстрировать обучение, прогноз, экономическую оценку, super(), вывод __mro__, обработку исключений и проверку типов. Для профиля ИИ показать метрики качества и подбор гиперпараметра. Построить UML-диаграммы классов и последовательности сценария создания объекта, загрузки данных, обучения, расчёта индикатора и экспорта отчёта.

Теоретические сведения

Множественное наследование позволяет классу иметь несколько родителей. Python определяет порядок поиска методов с помощью C3-линеаризации: он доступен через Class.__mro__ и Class.mro(). Порядок

сохраняет приоритет родителей и согласованность уже построенных иерархий. Каждый класс встречается в MRO один раз.

Миксин добавляет отдельную функциональность и используется вместе с основным классом. Он может не иметь состояния, как ValidationMixin, или хранить служебное состояние, как кэш и журнал. super() передаёт вызов следующему подходящему классу в MRO фактического объекта. Это особенно важно при ромбовидной иерархии: согласованные сигнатуры и однократный вызов super() предотвращают повторную инициализацию общего предка.

Ход работы

1. Сотрудники компании

Я реализовала Employee с общим счётчиком идентификаторов. ManagerSkills.work возвращает строку «id-й управляет», DeveloperSkills.work — «id-й программирует». Классы Manager и Developer объединяют состояние Employee с нужным поведением. Создала список из четырёх сотрудников и вызвала work для каждого элемента (см. рисунки 1, 2). Идентификаторы уникальны среди созданных сотрудников в пределах одного запуска программы.

В исходных Manager и Developer конфликта work нет: Employee этот метод не определяет. Потенциальная неоднозначность возникает при объединении ManagerSkills и DeveloperSkills в одном классе. Без переопределения Python выбирает первый подходящий метод по MRO. Для TeamLead я явно переопределила work и вызвала оба метода родителей, объединив результаты. Такой прямой вызов подходит для независимых методов; для кооперативной цепочки инициализации в аналитическом ядре используется super().

```
Сотрудники компании
1-й управляет
2-й программирует
3-й управляет
4-й программирует
MRO Manager: Manager
  → Employee
  → ManagerSkills
  → object
MRO Developer: Developer
  → Employee
  → DeveloperSkills
  → object
Совмещение обязанностей: 5-й управляет; 5-й программирует
MRO TeamLead: TeamLead
  → Employee
  → ManagerSkills
  → DeveloperSkills
  → object
```

Рисунок 1 – Результат работы программы «Сотрудники компании»

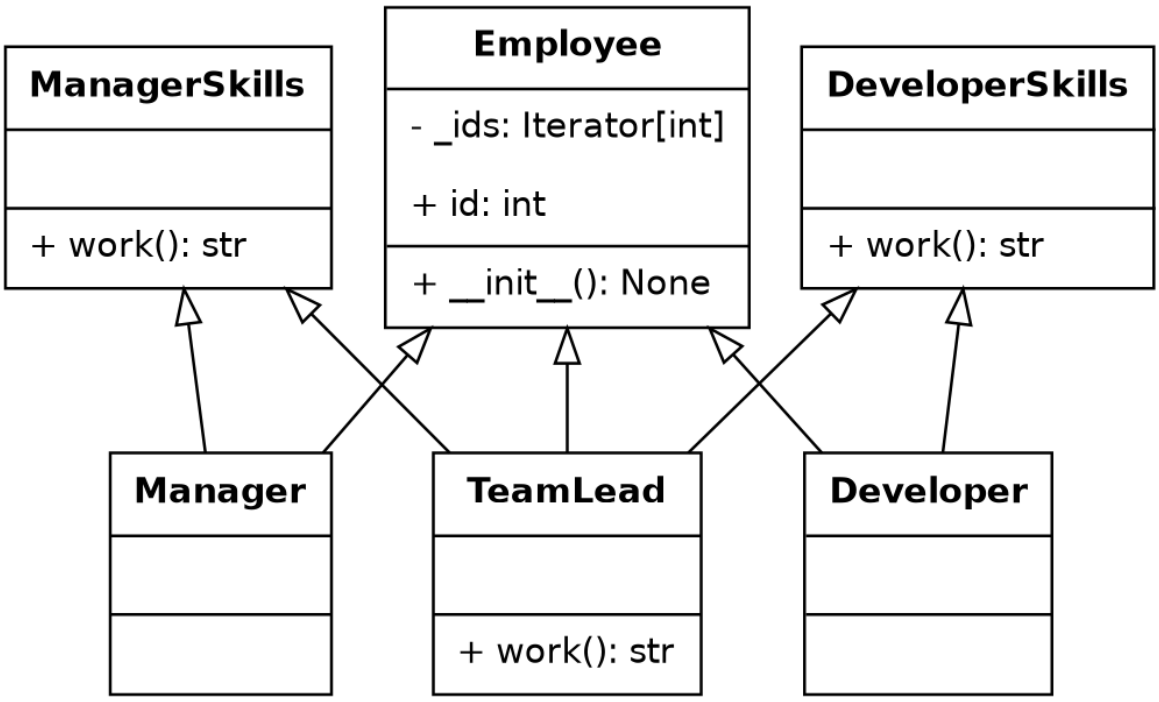


Рисунок 2 – UML-диаграмма классов сотрудников компании

Листинг 1 – Код программы

```
"""Задание 1. Вариант 2: сотрудники компании."""
from itertools import count


class Employee:
    _ids = count(1)

    def __init__(self):
        self.id = next(Employee._ids)


class ManagerSkills:
    def work(self):
        return f"{self.id}-й управляет"


class DeveloperSkills:
    def work(self):
        return f"{self.id}-й программирует"


class Manager(Employee, ManagerSkills):
    pass


class Developer(Employee, DeveloperSkills):
    pass


class TeamLead(Employee, ManagerSkills, DeveloperSkills):
    def work(self):
        # Явно объединяем одноимённые методы двух родителей.
        return (ManagerSkills.work(self) + "; "
                + DeveloperSkills.work(self))


def main():
    group = [Manager(), Developer(), Manager(), Developer()]
    print("Сотрудники компании")
    for employee in group:
        print(employee.work())
    print("MRO Manager:",
          " -> ".join(c.__name__ for c in Manager.__mro__))
    print("MRO Developer:",
          " -> ".join(c.__name__ for c in Developer.__mro__))
    lead = TeamLead()
    print("Совмещение обязанностей:", lead.work())
    print("MRO TeamLead:",
          " -> ".join(c.__name__ for c in TeamLead.__mro__))
```

```
if __name__ == "__main__":
    main()
```

2. Слои нейросети и информационно-аналитическое ядро

2.1. Слои нейросети — вариант 2

Я реализовала Layer с уникальным id, миксины поведения DenseLayer и ConvolutionalLayer, а также классы Dense(Layer, DenseLayer) и Conv(Layer, ConvolutionalLayer). Для каждого экземпляра вызываются forward и backward, выводится MRO (см. рисунки 3, 4). Оба класса поведения предоставляют оба прохода. В соответствии с вариантом методы возвращают строки: численные операции с тензорами и обратное распространение градиента в этом учебном примере не реализуются.

MRO для Dense: Dense → Layer → DenseLayer → object. Для Conv: Conv → Layer → ConvolutionalLayer → object. Конструктор находится в Layer, методы проходов — в соответствующем классе поведения. Таким образом, интерфейс объектов одинаков, а источник унаследованной реализации различается.

```
Слои нейросети
Dense: 1-й прямой проход
Dense: 1-й обратный проход
MRO: Dense -> Layer -> DenseLayer -> object
Conv: 2-й прямой проход
Conv: 2-й обратный проход
MRO: Conv -> Layer -> ConvolutionalLayer -> object
```

Рисунок 3 – Результат работы программы «Слои нейросети»

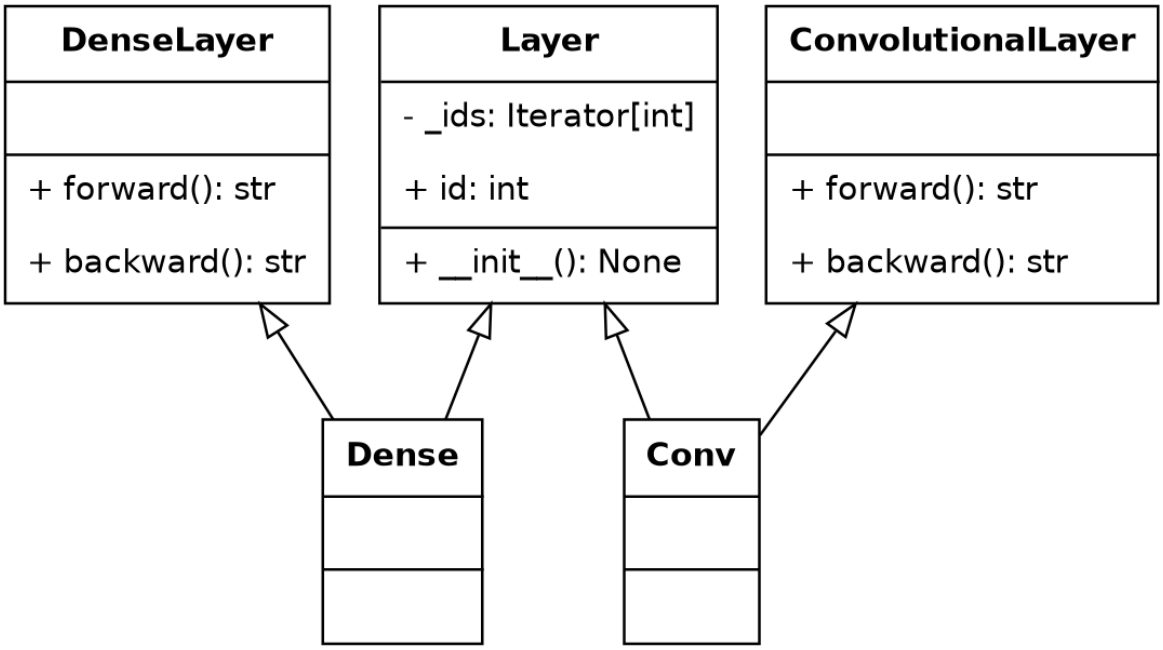


Рисунок 4 – UML-диаграмма классов Layer, Dense и Conv

Листинг 2 – Код программы

```
"""Задание 2. Вариант 2 для ИИ: слои нейросети."""
from itertools import count


class Layer:
    _ids = count(1)

    def __init__(self):
        self.id = next(Layer._ids)


class DenseLayer:
    def forward(self):
        return f"{self.id}-й прямой проход"

    def backward(self):
        return f"{self.id}-й обратный проход"


class ConvolutionalLayer:
    def forward(self):
        return f"{self.id}-й прямой проход"

    def backward(self):
        return f"{self.id}-й обратный проход"


class Dense(Layer, DenseLayer):
    pass
```

```
class Conv(Layer, ConvolutionalLayer):
    pass


def main():
    print("Слои нейросети")
    layers = [Dense(), Conv()]
    for layer in layers:
        print(type(layer).__name__ + ": " + layer.forward())
        print(type(layer).__name__ + ": " + layer.backward())
        print("MRO:", " -> ".join(
            c.__name__ for c in type(layer).__mro__))


if __name__ == "__main__":
    main()
```

2.2. Архитектура аналитического ядра

Чтобы выполнить общую часть задания 2, я разработала отдельный модуль analytics_core.py. BaseComponent создаёт общий уникальный id и копирует metadata. ValidationMixin проверяет числовые данные; CachingMixin повторно использует прогнозы; LoggingMixin сохраняет операции в списке events; ReportMixin экспортирует словарь в JSON-строку и, по указанному пути, в UTF-8-файл. Служебное состояние кэша и журнала создаётся кооперативными конструкторами.

MLModel реализует одномерную kNN-регрессию: хранит обучающие пары, выбирает k ближайших по абсолютному расстоянию значений x и возвращает среднее соответствующих y. При равных расстояниях выбирается более ранняя запись. Метод tune_k сравнивает кандидаты с помощью leave-one-out: каждая точка по очереди исключается из обучения и используется только для проверки. Критерий — минимум среднеквадратичной ошибки MSE; при равенстве выбирается меньшее k. После подбора модель обучается на всех данных с лучшим k.

Метод metrics вычисляет $MAE = \Sigma|\hat{y}_i - y_i| / n$ и $RMSE = \sqrt{(\Sigma(\hat{y}_i - y_i)^2 / n)}$. В демонстрации эти две метрики вычисляются на обучающем наборе и характеризуют подгонку, а не качество на независимой тестовой выборке. При

fit и load_data кэш очищается; после загрузки новых данных требуется повторное обучение.

EconomicIndicator вычисляет ROI = (R − C) / C · 100 %, где R — выручка, C — положительные затраты. Положительный ROI интерпретируется как прибыль, отрицательный — как убыток, нулевой — как безубыточность. HybridAnalyzer наследует MLModel и EconomicIndicator, получает прогноз выручки и передаёт его в экономический расчёт. Это учебный сценарий оценки по прогнозу; модель kNN не экстраполирует тренд за пределы обучающего диапазона.

2.3. MRO, super() и конфликты методов

В программе создаются отдельные экземпляры MLModel и EconomicIndicator, затем HybridAnalyzer. Фактический MRO гибридного класса: HybridAnalyzer → MLModel → EconomicIndicator → ValidationMixin → CachingMixin → LoggingMixin → ReportMixin → BaseComponent → object (см. рисунок 5). Один набор общих миксинов и BaseComponent включается в порядок поиска только один раз, несмотря на несколько путей наследования.

В конструкторах сначала вызывается super().__init__, после возврата добавляется запись в init_trace. Поэтому список завершения инициализации идёт от общего предка к потомку: BaseComponent → LoggingMixin → CachingMixin → EconomicIndicator → MLModel → HybridAnalyzer. Это порядок завершения кооперативных конструкторов, а не новая последовательность MRO.

Метод describe переопределён в обоих основных родителях. MLModel.describe вызывает super().describe(), что для объекта HybridAnalyzer приводит к EconomicIndicator.describe, затем к BaseComponent.describe. Результат: «Компонент 3 / EconomicIndicator / MLModel / HybridAnalyzer». Это демонстрирует, что super() следует MRO фактического объекта и не означает просто вызов текстуально указанного родителя.

Если разные родители реализуют validate или log независимо, победит первый подходящий метод по MRO; важная проверка может быть пропущена.

Я вынесла общие операции в единые миксины. При необходимости можно переопределить конфликтующий метод, явно вызвать нужный класс либо изменить порядок родителей. Для кооперативной работы все участники цепочки должны сохранять совместимые сигнатуры, передавать оставшиеся **kwargs и использовать super(). Прямые вызовы двух родительских конструкторов могут повторно инициализировать общего предка.

Несогласуемый порядок родителей Python отклоняет с TypeError уже при создании класса: например, X(A, B) и Y(B, A) невозможно объединить в Z(X, Y). Этот случай отдельно проверен тестом. Перегрузка арифметических операторов в данной работе не требуется. В переопределённых методах сохранены совместимые сигнатуры; входные данные проверяются по типу и диапазону.

```
MLModel, прогноз для 2: 200.0
EconomicIndicator, ROI: 50.0
MRO HybridAnalyzer:
  HybridAnalyzer
  MLModel
  EconomicIndicator
  ValidationMixin
  CachingMixin
  LoggingMixin
  ReportMixin
  BaseComponent
  object
Завершение конструкторов (без повторов):
  BaseComponent
  LoggingMixin
  CachingMixin
  EconomicIndicator
  MLModel
  HybridAnalyzer
super(): Компонент 3 / EconomicIndicator / MLModel / HybridAnalyzer
```

Рисунок 5 – Создание компонентов, фактический MRO и работа super()

2.4. Контрольный пример и UML-моделирование

Я загрузила x = [1, 2, 3, 4, 5] и y = [100, 200, 300, 400, 500]. LOO MSE для k = 1 равна 10000; для k = 2 — 9000; для k = 3 — 18666,67. Выбрано k = 2. На обучающих данных MAE = 50 и RMSE = 50. Для x = 6 ближайшие значения y

равны 500 и 400: прогноз (500 + 400) / 2 = 450. При затратах 300 получен ROI = (450 − 300) / 300 · 100 % = 50 % (см. рисунок 6). Повторный прогноз 450 берётся из кэша.

Результат экспортирован методом export_report в файл example_report.json, включённый в комплект работы. В демонстрации main JSON выводится в консоль; при генерации иллюстраций тем же методом записывается отдельный файл. Идентификатор зависит от порядка создания объектов и не влияет на вычисления.

```
Подбор k по LOO MSE:
  k=1: MSE=10000.00
  k=2: MSE=9000.00
  k=3: MSE=18666.67
Выбрано k = 2
Метрики на обучающих данных: {'MAE': 50.0, 'RMSE': 50.0}
Повторный прогноз (кэш): 450.0
JSON-отчёт:
{
  "id": 3,
  "metadata": {
    "author": "Шевченко О.Н."
  },
  "k": 2,
  "predicted_revenue": 450.0,
  "cost": 300,
  "ROI_percent": 50.0,
  "interpretation": "Прибыль"
}
```

Рисунок 6 – Подбор k, метрики, прогноз и JSON-отчёт

Я построила UML-диаграммы с именами классов, полями, методами, типами и видимостью: «+» — открытый член, «−» — внутренний член по соглашению Python. Полные описания миксинов приведены на рисунке 7, основных классов — на рисунке 8. На рисунке 8 показана основная иерархия с общим предком; все прямые связи с миксинами отдельно показаны на рисунке

9. Пустой треугольник направлен к родителю. Композиция классов в этом ядре не используется.

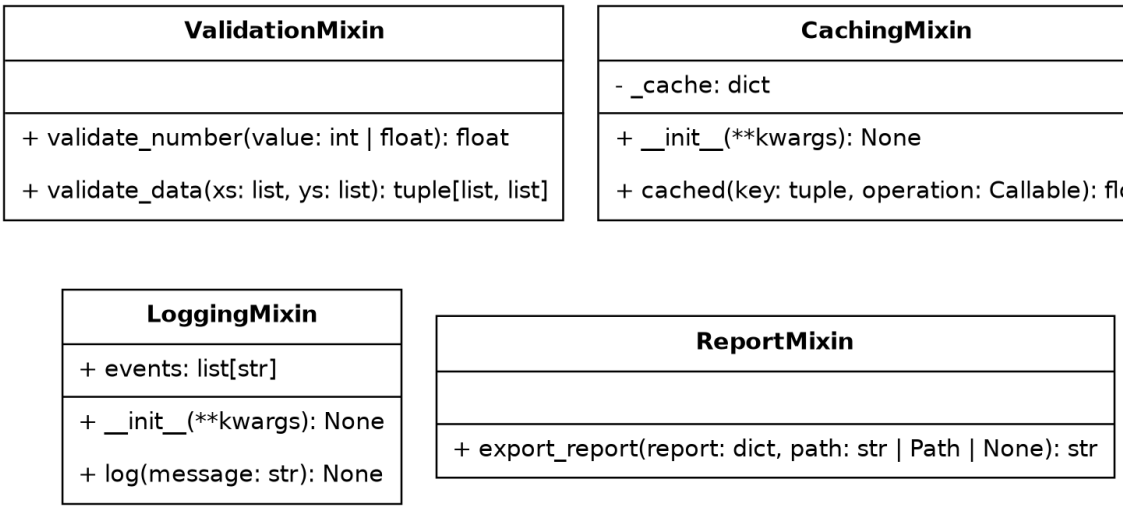


Рисунок 7 – UML-описания миксинов аналитического ядра

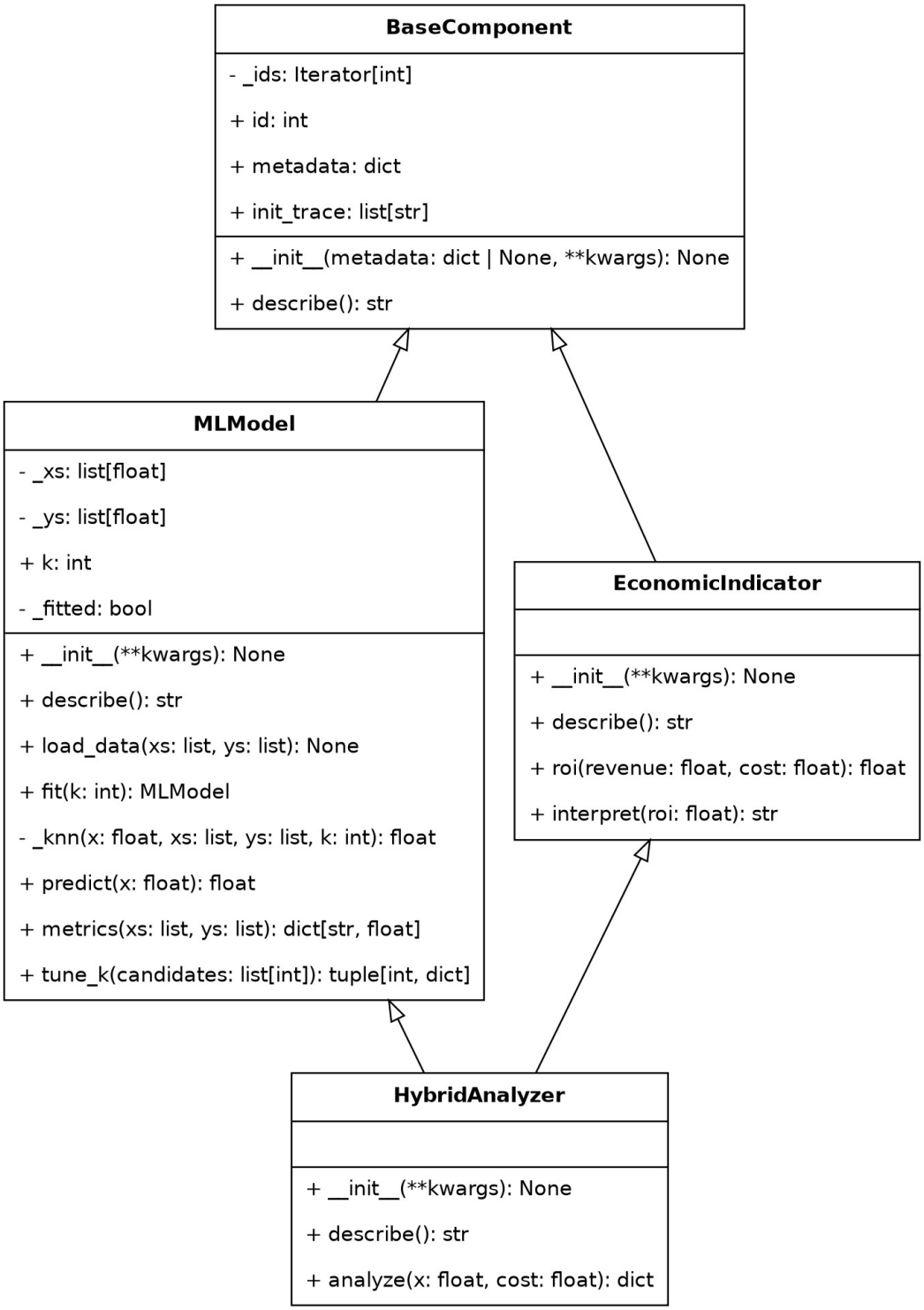


Рисунок 8 – UML-описания основных классов и общего предка

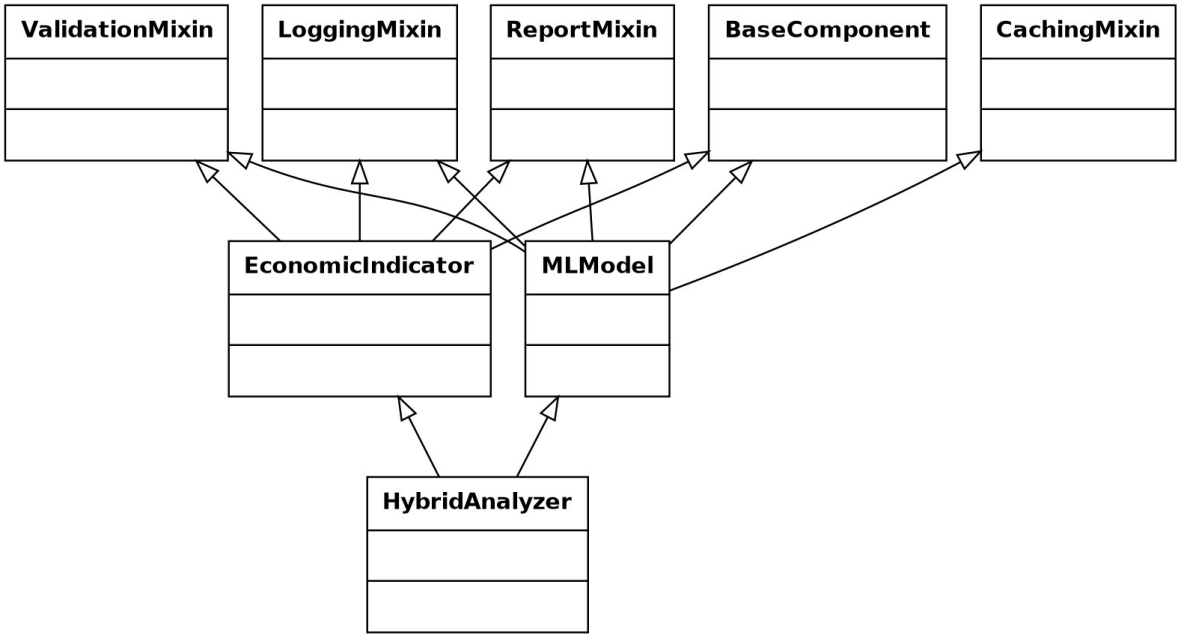


Рисунок 9 – Полная схема прямых связей наследования аналитического ядра

Диаграмма последовательности показывает создание объекта, кооперативную инициализацию, загрузку и проверку данных, подбор k с обучением, получение прогноза, расчёт ROI и экспорт в JSON-файл (см. рисунок 10). Вызовы родителей объединены в одну линию жизни для читаемости.

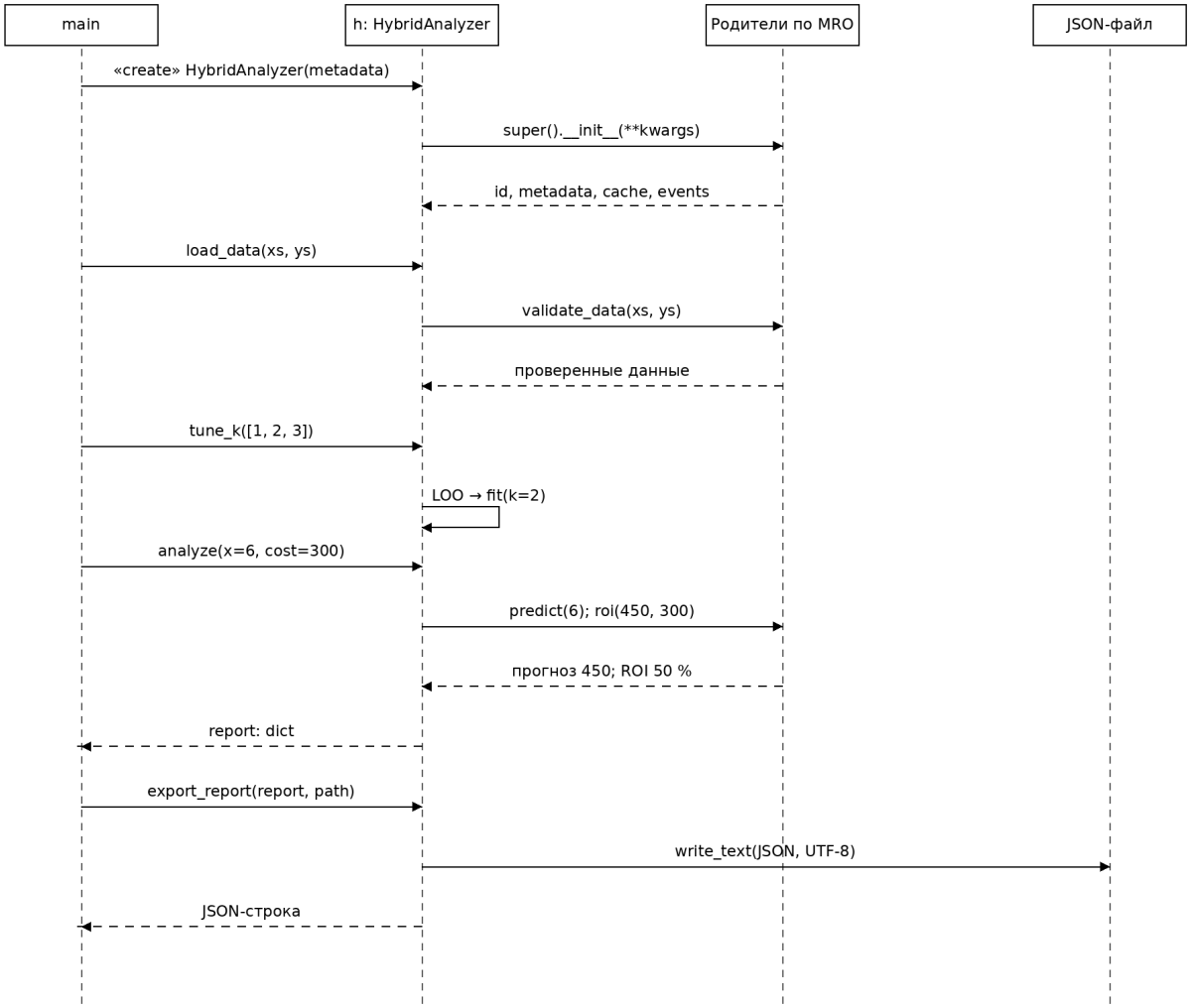


Рисунок 10 – Диаграмма последовательности аналитического сценария

В журнале видны операции load data, fit, tune, cache miss, cache hit, calculate ROI и export JSON. При нулевых затратах возбуждается ZeroDivisionError, который перехватывается в демонстрации (см. рисунок 11). Валидация также отклоняет строки вместо чисел, bool, NaN, бесконечность, неравные длины наборов, недопустимые k и отрицательные затраты. Прогноз до обучения приводит к RuntimeError.

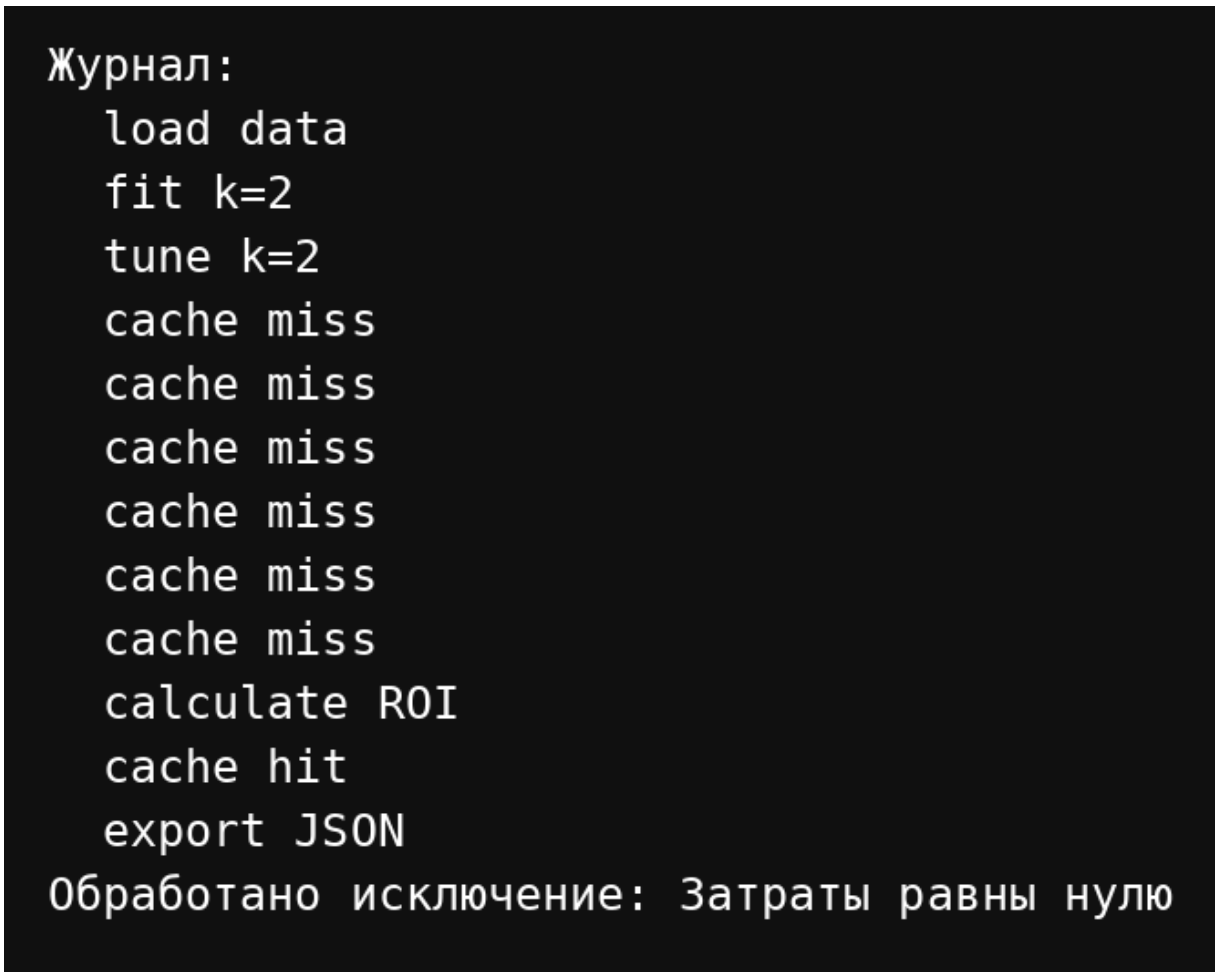


Рисунок 11 – Журналирование и обработка деления на ноль

Листинг 3 – Код программы

```
"""Общие требования задания 2: аналитическое ядро и MRO."""
import json
from itertools import count
from math import isfinite, sqrt
from pathlib import Path


class BaseComponent:
    _ids = count(1)

    def __init__(self, metadata=None, **kwargs):
        super().__init__(**kwargs)
        if (metadata is not None
                and not isinstance(metadata, dict)):
            raise TypeError("metadata должен быть словарём")
        self.id = next(BaseComponent._ids)
        self.metadata = dict(metadata or {})
        self.init_trace = ["BaseComponent"]

    def describe(self):
        return f"Компонент {self.id}"
```

```
class ValidationMixin:
    @staticmethod
    def validate_number(value):
        if type(value) not in (int, float):
            raise TypeError("Ожидается число, кроме bool")
        if not isfinite(value):
            raise ValueError("Число должно быть конечным")
        return float(value)

    def validate_data(self, xs, ys):
        if not isinstance(xs, (list, tuple)) or not isinstance(
                ys, (list, tuple)):
            raise TypeError("Данные должны быть списками")
        if len(xs) != len(ys) or len(xs) < 2:
            raise ValueError("Нужно >= 2 пары одинаковой длины")
        return ([self.validate_number(x) for x in xs],
                [self.validate_number(y) for y in ys])


class CachingMixin:
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self._cache = {}
        self.init_trace.append("CachingMixin")

    def cached(self, key, operation):
        if key in self._cache:
            self.log("cache hit")
            return self._cache[key]
        self._cache[key] = operation()
        self.log("cache miss")
        return self._cache[key]


class LoggingMixin:
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.events = []
        self.init_trace.append("LoggingMixin")

    def log(self, message):
        if not isinstance(message, str):
            raise TypeError("Сообщение должно быть строкой")
        self.events.append(message)


class ReportMixin:
    def export_report(self, report, path=None):
        if not isinstance(report, dict):
            raise TypeError("Отчёт должен быть словарём")
        text = json.dumps(report, ensure_ascii=False,
                          indent=2, allow_nan=False)
```

```
        if path is not None:
            Path(path).write_text(text, encoding="utf-8")
        self.log("export JSON")
        return text


class MLModel(ValidationMixin, CachingMixin, LoggingMixin,
              ReportMixin, BaseComponent):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self._xs, self._ys = [], []
        self.k = 1
        self._fitted = False
        self.init_trace.append("MLModel")

    def describe(self):
        return super().describe() + " / MLModel"

    def load_data(self, xs, ys):
        xs, ys = self.validate_data(xs, ys)
        self._xs, self._ys = xs, ys
        self._fitted = False
        self._cache.clear()
        self.log("load data")

    def fit(self, k=1):
        if type(k) is not int:
            raise TypeError("k должен быть целым числом")
        if not 1 <= k <= len(self._xs):
            raise ValueError("k вне диапазона [1, число точек]")
        self.k = k
        self._fitted = True
        self._cache.clear()
        self.log(f"fit k={k}")
        return self

    @staticmethod
    def _knn(x, xs, ys, k):
        indices = sorted(range(len(xs)),
                         key=lambda i: (abs(xs[i] - x), i))[:k]
        return sum(ys[i] for i in indices) / k

    def predict(self, x):
        if not self._fitted:
            raise RuntimeError("Сначала обучите модель")
        x = self.validate_number(x)
        return self.cached((self.k, x), lambda: self._knn(
            x, self._xs, self._ys, self.k))

    def metrics(self, xs, ys):
        xs, ys = self.validate_data(xs, ys)
        errors = [self.predict(x) - y for x, y in zip(xs, ys)]
        return {"MAE": sum(abs(e) for e in errors) / len(errors),
```

```
                "RMSE": sqrt(sum(e * e for e in errors)
                             / len(errors))}

    def tune_k(self, candidates):
        if not isinstance(candidates, (list, tuple)):
            raise TypeError("Кандидаты k должны быть списком")
        if not candidates or len(self._xs) < 2:
            raise ValueError("Нет данных или кандидатов")
        scores = {}
        for k in candidates:
            if type(k) is not int or not 1 <= k < len(self._xs):
                raise ValueError("Для LOO требуется 1 <= k < n")
            errors = []
            for i, x in enumerate(self._xs):
                xs = self._xs[:i] + self._xs[i + 1:]
                ys = self._ys[:i] + self._ys[i + 1:]
                prediction = self._knn(x, xs, ys, k)
                errors.append((prediction - self._ys[i]) ** 2)
            scores[k] = sum(errors) / len(errors)
        best = min(scores, key=lambda k: (scores[k], k))
        self.fit(best)
        self.log(f"tune k={best}")
        return best, scores


class EconomicIndicator(ValidationMixin, LoggingMixin,
                        ReportMixin, BaseComponent):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.init_trace.append("EconomicIndicator")

    def describe(self):
        return super().describe() + " / EconomicIndicator"

    def roi(self, revenue, cost):
        revenue = self.validate_number(revenue)
        cost = self.validate_number(cost)
        if cost == 0:
            raise ZeroDivisionError("Затраты равны нулю")
        if cost < 0 or revenue < 0:
            raise ValueError("Выручка и затраты должны быть >= 0")
        result = (revenue - cost) / cost * 100
        self.log("calculate ROI")
        return result

    def interpret(self, roi):
        roi = self.validate_number(roi)
        if roi > 0:
            return "Прибыль"
        if roi < 0:
            return "Убыток"
        return "Безубыточность"
```

```
class HybridAnalyzer(MLModel, EconomicIndicator):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.init_trace.append("HybridAnalyzer")

    def describe(self):
        return super().describe() + " / HybridAnalyzer"

    def analyze(self, x, cost):
        revenue = self.predict(x)
        roi = self.roi(revenue, cost)
        return {"id": self.id, "metadata": self.metadata,
                "k": self.k, "predicted_revenue": revenue,
                "cost": cost, "ROI_percent": roi,
                "interpretation": self.interpret(roi)}


def main():
    model = MLModel(metadata={"role": "ML"})
    model.load_data([1, 2, 3], [100, 200, 300])
    model.fit()
    print("MLModel, прогноз для 2:", model.predict(2))
    indicator = EconomicIndicator(metadata={"role": "economy"})
    print("EconomicIndicator, ROI:", indicator.roi(450, 300))
    hybrid = HybridAnalyzer(metadata={"author": "Шевченко О.Н."})
    print("MRO HybridAnalyzer:")
    for cls in type(hybrid).__mro__:
        print("  " + cls.__name__)
    print("Завершение конструкторов (без повторов):")
    for name in hybrid.init_trace:
        print("  " + name)
    print("super():", hybrid.describe())
    hybrid.load_data([1, 2, 3, 4, 5], [100, 200, 300, 400, 500])
    best, scores = hybrid.tune_k([1, 2, 3])
    print("Подбор k по LOO MSE:")
    for k, mse in scores.items():
        print(f"  k={k}: MSE={mse:.2f}")
    print("Выбрано k =", best)
    print("Метрики на обучающих данных:", hybrid.metrics(
        [1, 2, 3, 4, 5], [100, 200, 300, 400, 500]))
    report = hybrid.analyze(6, 300)
    print("Повторный прогноз (кэш):", hybrid.predict(6))
    print("JSON-отчёт:\n" + hybrid.export_report(report))
    print("Журнал:")
    for event in hybrid.events:
        print("  " + event)
    try:
        hybrid.roi(450, 0)
    except ZeroDivisionError as error:
        print("Обработано исключение:", error)
```

```
if __name__ == "__main__":
    main()
```

Проверка результатов

Я выполнила 14 автоматических тестов unittest — все завершились успешно. Проверены смешанная группа сотрудников, уникальность id, явное разрешение конфликта work и изменение приоритета родителей; проходы Dense и Conv; фактический MRO и однократная инициализация BaseComponent; прогноз kNN и разрешение равных расстояний; подбор k по LOO без включения проверяемой точки; MAE и RMSE; срабатывание и очистка кэша; ROI и его интерпретация; полный сценарий с записью и чтением JSON; проверки типов, диапазонов и несовместимый MRO. Все три программы также запущены отдельно как консольные приложения. Полные протоколы запусков и результаты проверок приложены к исходникам.

Для воспроизведения из каталога lab3_shevchenko: python3 -m unittest discover -s programs -v. Программы используют только стандартную библиотеку Python; установка ML-фреймворков не требуется.

Вывод: я освоила принципы множественного наследования в Python, изучила механизм MRO и работу super(). Реализовала классы сотрудников и слоёв нейросети для второго варианта, разработала аналитическое ядро с миксинами проверки данных, кэширования, журналирования и экспорта. Выполнила обучение kNN-регрессора, подбор гиперпараметра, оценку ошибок и экономический расчёт. Научилась разрешать конфликты одноимённых методов и предотвращать повторную инициализацию общего предка. Построила UML-диаграммы классов и последовательности, проверила решения контрольными примерами и автоматическими тестами.